

2-Bit Arithmetic Logic Unit (ALU)

Comprehensive Technical Design, Theory, Schematics & Verilog Implementation

1. Conceptual Overview: What is an ALU?

An Arithmetic Logic Unit (ALU) serves as the computational heart of any central processing unit (CPU). It takes two binary inputs (operands), executes a specific mathematical or logical task designated by control signals (selection lines), and outputs both the computed data word and hardware status indicators (such as carry or borrow).

In this 2-bit ALU architecture:

- Operands (A and B):** Two 2-bit unsigned binary numbers: $A = [A1, A0]$ and $B = [B1, B0]$. Each operand spans values from 0 to 3 in decimal (00, 01, 10, 11 in binary).
- Operation Selector (Sel):** A 2-bit control code $Sel = [Sel1, Sel0]$. Since 2 bits offer $2^2 = 4$ permutations, it switches between four fundamental operations: Addition, Subtraction, Bitwise AND, and Bitwise OR.
- Data Output (ALU_Out):** A 2-bit output vector $[Y1, Y0]$ representing the calculated result of the selected function.
- Status Flag (Carry):** A 1-bit indicator reporting arithmetic overflow (during addition) or borrow conditions (during subtraction). It is actively grounded to 0 during logical operations.

2. ALU Truth Table & Functional Modes

The function executed by the ALU is strictly determined by the 2-bit selection input $Sel[1:0]$:

Sel [1:0]	Mode	Operation	ALU_Out Expression	Carry / Borrow Flag
00	Arithmetic	Addition (ADD)	$A + B$	1 if Sum > 3 (Overflow)
01	Arithmetic	Subtraction (SUB)	$A - B$	1 if $A < B$ (Borrow)
10	Logic	Bitwise AND	$A \& B$	0 (Deasserted)
11	Logic	Bitwise OR	$A B$	0 (Deasserted)

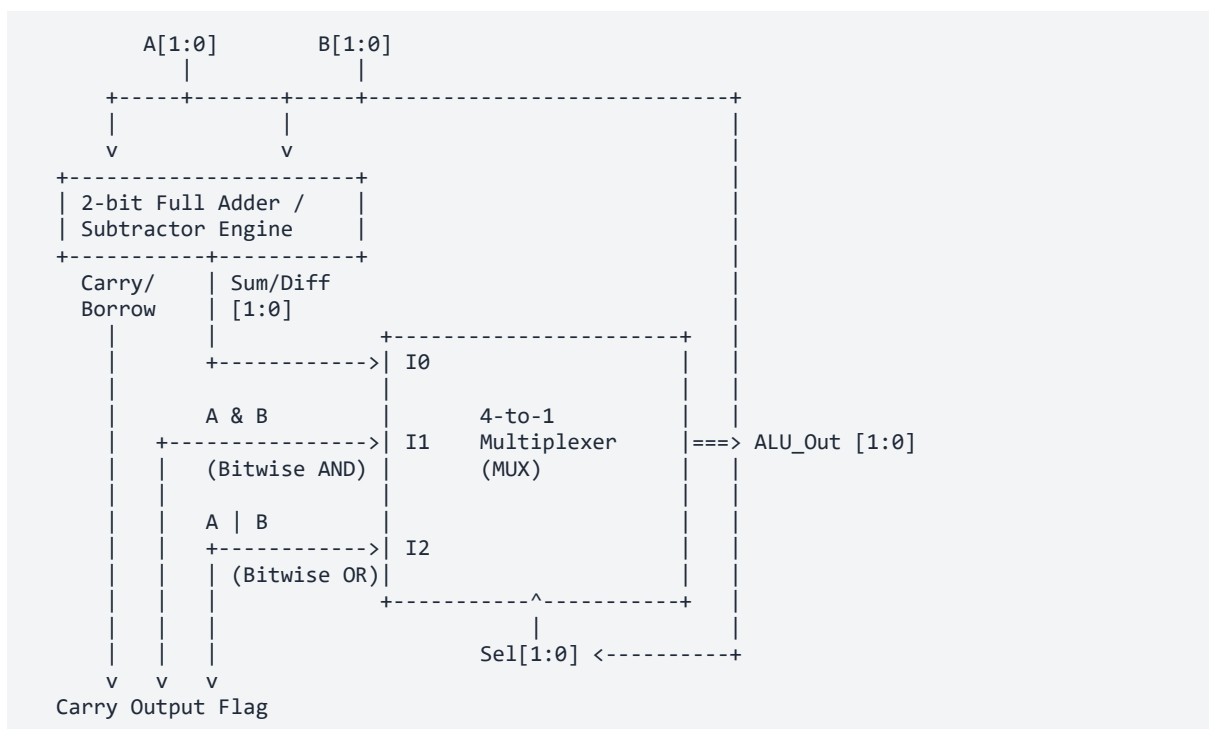
Illustrative Input-Output Combinations:

Sel	A (Bin / Dec)	B (Bin / Dec)	Function	Output (Bin / Dec)	Carry	Hardware Significance
00	10 (2)	01 (1)	ADD: $2 + 1$	11 (3)	0	Exact representation; no overflow.

00	11 (3)	10 (2)	ADD: 3 + 2	01 (1)	1	Total is 5 (101 ₂). Overflow captured by Carry.
01	11 (3)	01 (1)	SUB: 3 - 1	10 (2)	0	Positive difference; no borrow generated.
01	01 (1)	10 (2)	SUB: 1 - 2	11 (3)	1	Negative result in 2's comp; Borrow=1.
10	11 (3)	10 (2)	AND: 11 & 10	10 (2)	0	Bitwise mask extraction; Carry cleared.
11	01 (1)	10 (2)	OR: 01 10	11 (3)	0	Bitwise set union; Carry cleared.

3. Hardware Architecture & Structural Schematic

In modern digital hardware, ALUs do not execute sequentially through software loops. Instead, all specialized compute engines operate in parallel the moment inputs A and B stabilize. A 4-to-1 Multiplexer (MUX) serves as the routing switch, steering the appropriate calculated result to the primary output pins:



4. In-Depth: Role & Handling of the Carry Flag

A common engineering question is: What happens to the Carry output once calculated? In digital systems, its handling depends directly on the system hierarchy:

- **Lab & FPGA Prototyping:** The Carry line is typically mapped to an onboard physical LED or combined into a 3-bit compound bus {Carry, ALU_Out}. For example, when adding 3 + 2, the bus yields binary 101 (decimal 5), ensuring no mathematical data is truncated.
- **CPU Architecture (Status/Flags Register):** In complete microprocessors, the Carry signal is captured in a dedicated 1-bit D Flip-Flop known as the Carry Flag (CF) within the Program Status Word (PSW). This stored bit enables two vital CPU functions: (1) Conditional branch instructions such as JC (Jump if Carry) and JNC, which guide decision-making loops, and (2) Multi-precision cascading arithmetic via instructions like ADC (Add with Carry), where the saved Carry from byte 0 cascades into the Cin of byte 1.
- **Logical Mode Isolation:** During AND/OR evaluations, bit operations are purely vertical and independent. Leaving the Carry line in an undefined or floating state could corrupt downstream logic. Therefore, behavioral Verilog design dictates that Carry is explicitly reset to 1'b0 during all logical instructions.

5. Complete Verilog Implementation (Behavioral Model)

The following synthesizable Verilog code utilizes procedural assignments inside an always @(*) sensitivity block to execute the arithmetic and logical selections without inferred latches:

```
// =====  
// Module: alu_2bit  
// Description: Synthesizable 2-Bit Behavioral Arithmetic Logic Unit  
// =====  
`timescale 1ns / 1ps  
  
module alu_2bit (  
    input wire [1:0] A,          // 2-bit Operand A  
    input wire [1:0] B,          // 2-bit Operand B  
    input wire [1:0] Sel,        // Mode Select: 00:ADD, 01:SUB, 10:AND, 11:OR  
    output reg [1:0] ALU_Out,    // 2-bit Operation Result  
    output reg      Carry        // Arithmetic Overflow / Borrow Flag  
);  
  
    always @(*) begin  
        // Default assignments guarantee latch-free combinational logic  
        ALU_Out = 2'b00;  
        Carry   = 1'b0;  
  
        case (Sel)  
            2'b00: begin // ADDITION (A + B)  
                // Concatenation captures the 3rd MSB as Carry  
                {Carry, ALU_Out} = A + B;  
            end  
  
            2'b01: begin // SUBTRACTION (A - B)  
                // Carry bit reflects borrow status if A < B  
                {Carry, ALU_Out} = A - B;  
            end  
  
            2'b10: begin // BITWISE AND  
                ALU_Out = A & B;  
                Carry   = 1'b0; // Explicitly cleared  
            end  
  
            2'b11: begin // BITWISE OR  
                ALU_Out = A | B;  
                Carry   = 1'b0; // Explicitly cleared  
            end  
        end  
    end
```

```
        default: begin
            ALU_Out = 2'b00;
            Carry    = 1'b0;
        end
    endcase
end

endmodule
```

6. Quick Revision Summary

Key Takeaways for Exams and Interviews:

1. Number of Operations: 2 select lines yield $2^2 = 4$ operations (ADD, SUB, AND, OR).
2. Parallel Computing: Internal hardware units (Adders, AND array, OR array) evaluate simultaneously; the Multiplexer isolates and routes the desired result.
3. Arithmetic vs. Logic: Arithmetic commands generate dynamic carry/borrow flags; logic commands evaluate bit-by-bit independently with Carry forced to 0.
4. Concatenation Operator: In Verilog, {Carry, ALU_Out} = A + B elegantly packs the 3-bit arithmetic output into the separate 1-bit flag and 2-bit data bus.